Système et Réseaux
CM11
DNS, IPv6 et `getaddrinfo`

Pierre Rousselin
rousselin@univ-paris13.fr

Université Sorbonne Paris Nord
L2 informatique et double-licence

mai 2025

Le DNS

Couche réseau : IPv6

Fonction `getaddrinfo`

Le DNS

Couche réseau : IPv6

Fonction `getaddrinfo`

Motivation : rappel sur la couche transport

Pour qu'un processus A sur un hôte H_A puisse envoyer un message à un processus B sur un hôte H_B, il faut :

- Que A et B utilisent le même protocole transport (UDP ou bien TCP) ;

Motivation : rappel sur la couche transport

Pour qu'un processus A sur un hôte H_A puisse envoyer un message à un processus B sur un hôte H_B, il faut :

- Que A et B utilisent le même protocole transport (UDP ou bien TCP) ;
  ok : fixé par le protocole applicatif
- Que A connaisse *le port* utilisé par le processus B pour recevoir des messages ;

Motivation : rappel sur la couche transport

Pour qu'un processus A sur un hôte H_A puisse envoyer un message à un processus B sur un hôte H_B, il faut :

- Que A et B utilisent le même protocole transport (UDP ou bien TCP) ;
  ok : fixé par le protocole applicatif
- Que A connaisse *le port* utilisé par le processus B pour recevoir des messages ;
  ok : fixé (pour le serveur) par le protocole applicatif
- Que A connaisse l'adresse IP de l'hôte H_B dans le réseau ;

Motivation : rappel sur la couche transport

Pour qu'un processus A sur un hôte H_A puisse envoyer un message à un processus B sur un hôte H_B, il faut :

- Que A et B utilisent le même protocole transport (UDP ou bien TCP) ;
  ok : fixé par le protocole applicatif
- Que A connaisse *le port* utilisé par le processus B pour recevoir des messages ;
  ok : fixé (pour le serveur) par le protocole applicatif
- Que A connaisse l'adresse IP de l'hôte H_B dans le réseau ;
  C'est le rôle du DNS.

DNS ?

Figure – photo des pages blanches, source https://www.rtl.fr/actu/debats-societe/le-bottin-telephonique-n-est-plus-distribue-dans-certains-departements-7786776087 Crédit : DR (? ?)

DNS ?

Figure – photo des pages blanches, source https://www.rtl.fr/actu/debats-societe/le-bottin-telephonique-n-est-plus-distribue-dans-certains-departements-7786776087 Crédit : DR (? ?)

Oui et non.

Domain Name System

- Le DNS (pour *Domain Name System*) permet d'associer des noms de domaines (comme `galilee.univ-paris13.fr`) à une ou plusieurs adresses IP (comme `81.194.43.211` ou `81.194.43.212`).
- Un peu comme un annuaire associe des numéros de téléphones à des personnes...
- ... mais qui avait l'autorité sur le bon vieux bottin ?

Domain Name System

- Le DNS (pour *Domain Name System*) permet d'associer des noms de domaines (comme `galilee.univ-paris13.fr`) à une ou plusieurs adresses IP (comme `81.194.43.211` ou `81.194.43.212`).
- Un peu comme un annuaire associe des numéros de téléphones à des personnes...
- ... mais qui avait l'autorité sur le bon vieux bottin ?
- À ses débuts, les « Postes et Télégraphes », puis les PTT (postes, télégraphes et téléphones) et enfin France Télécom, qui avaient, sucessivement, le monopole de l'attribution des numéros de téléphones en France.

Le premier Bottin d'Internet : `HOSTS.TXT`

- fichier texte qui associe des noms à des adresses IP
- hébergé au *Stanford Research Institute*
- téléchargé régulièrement (chaque nuit) par chaque machine du réseau.

Problèmes :

Le premier Bottin d'Internet : `HOSTS.TXT`

- fichier texte qui associe des noms à des adresses IP
- hébergé au *Stanford Research Institute*
- téléchargé régulièrement (chaque nuit) par chaque machine du réseau.

Problèmes :

- Trop centralisé : si la machine de *Stanford* tombe en panne, tout le monde est dans le noir.
- Trop centralisé : un seul acteur (*Stanford*) a la main sur l'annuaire, comme pour l'ancien bottin français
- Ne passe pas à l'échelle : ne fonctionne pas si des dizaines de millions de machines demandent le fichier
- Trop rigide : les modifications mettent trop de temps à se propager
- Trop lourd : le fichier devient vite énorme

Le premier Bottin d'Internet : `HOSTS.TXT`

- fichier texte qui associe des noms à des adresses IP
- hébergé au *Stanford Research Institute*
- téléchargé régulièrement (chaque nuit) par chaque machine du réseau.

Problèmes :

- Trop centralisé : si la machine de *Stanford* tombe en panne, tout le monde est dans le noir.
- Trop centralisé : un seul acteur (*Stanford*) a la main sur l'annuaire, comme pour l'ancien bottin français
- Ne passe pas à l'échelle : ne fonctionne pas si des dizaines de millions de machines demandent le fichier
- Trop rigide : les modifications mettent trop de temps à se propager
- Trop lourd : le fichier devient vite énorme

Le DNS (*Stanford*, 1983) est apparu pour résoudre ces problèmes.

Nom de domaine

`galilee.univ-paris13.fr`

- Étiquette : composées de lettres (casse non prise en compte), chiffres et traits d'union, qui peut contenir de 1 à 63 octets.
  - `galilee`
  - `univ-paris13`
  - `fr`
- Nom de domaine : étiquettes séparées par des points.
- Longueur totale du nom de domaine ≤ 253.

Les domaines de premier niveau (*Top Level Domains*)

`galilee.univ-paris13.fr`

- On commence par la fin : le domaine de permier niveau (*Top Level Domain* ou TLD), ici `fr`.
- `fr` est un TLD national, comme `de`, `uk`, `us`, `cn`, `dz`, ...
- Il y a aussi des TLD génériques, comme `org` (en principe pour les organisation, souvent à but non lucratif), `net` (en principe pour les organisations liées à Internet), `com` (en principe pour *commercial*), ...

À la recherche de Galilée

Où es-tu ?

- À ce stade, on sait juste que `galilee.univ-paris13.fr` est en France (`fr`). `fr` devrait en savoir plus !
- Mais comment contacter `fr` ?

Les serveurs racine *root servers*

- Il existe 13 serveurs [1] DNS appelés *serveurs racines* (*root servers*).
- Leurs noms de domaines sont `a.root-servers.net`, `b.root-servers.net`, ..., `m.root-servers.net`.
- N'importe lequel d'entre eux devrait pouvoir m'indiquer l'adresse d'un serveur DNS qui connaît les adresses qui se terminent par `fr`.
- Problème : comment connaître l'adresse d'un *root server* ? Réponse un peu plus tard.

1. en fait ils sont répliqués de nombreuses fois, voir `https://root-servers.org`

Un délégataire du sous-domaine `fr` (1)

- On suppose que la commande `dig` connaît par magie des adresses IP de `a.root-servers.net`...

```
$ # demande à a.root-servers.net l'adresse de galilee.univ-paris13.fr
$ dig @a.root-servers.net galilee.univ-paris13.fr
; <<>> DiG 9.18.13 <<>> @a.root-servers.net fr
...
;; QUESTION SECTION:
;fr.                    IN        A
;; AUTHORITY SECTION:
fr.                172800        IN        NS        e.ext.nic.fr.
fr.                172800        IN        NS        f.ext.nic.fr.
...
;; ADDITIONAL SECTION:
e.ext.nic.fr. 172800        IN        A        193.176.144.22
```

- Réponse : les serveurs DNS (NS) qui font autorité sur `fr` sont `d.ext.nic.fr`, `e.ext.nic.fr`, ..., `g.ext.nic.fr`.
- Leurs adresses IP sont données dans la section additionnelle.

Un délégataire du sous-domaine `fr` (2)

- Plutôt que de donner directement l'adresse de `galiee.univ-paris13.fr`, les serveurs racines *délèguent* la résolution des noms de TLD `fr` à d'autres serveurs DNS.
- On a trouvé un délégataire pour `fr`, par exemple `e.ext.nic.fr`, d'adresse `193.176.144.22`
- Est-ce qu'il connaît ?

Ché pas, demande à eux

```
$ dig @193.176.144.22 galilee.univ-paris13.fr
; <<>> DiG 9.18.13 <<>> @193.176.144.22 galilee.univ-paris13.fr
;; QUESTION SECTION:
;galilee.univ-paris13.fr.        IN        A

;; AUTHORITY SECTION:
univ-paris13.fr.        3600        IN        NS        dns1.univ-paris13.fr.
univ-paris13.fr.        3600        IN        NS        dns2.univ-paris13.fr.
univ-paris13.fr.        3600        IN        NS        up13p7.univ-paris13.fr.

;; ADDITIONAL SECTION:
dns1.univ-paris13.fr.        3600        IN        A        193.54.26.1
dns2.univ-paris13.fr.        3600        IN        A        194.254.164.6
up13p7.univ-paris13.fr.        3600        IN        A        194.254.61.61
```

Il nous répond : « Je ne sais pas, mais je connais trois serveurs DNS qui font autorité sur `univ-paris13.fr`, je leur ai délégué cette autorité ».

On va te trouver !

```
$ dig @194.254.61.61 galilee.univ-paris13.fr
```

On va te trouver !

```
$ dig @194.254.61.61 galilee.univ-paris13.fr
;; communications error to 194.254.61.61#53: timed out
```

C'est normal à Paris 13 ? !

On va te trouver !

```
$ dig @194.254.61.61 galilee.univ-paris13.fr
;; communications error to 194.254.61.61#53: timed out
```

C'est normal à Paris 13 ? !

```
$ dig @193.54.26.1 galilee.univ-paris13.fr
; <<>> DiG 9.18.13 <<>> @193.54.26.1 galilee.univ-paris13.fr
;; QUESTION SECTION:
;galilee.univ-paris13.fr.        IN        A
;; ANSWER SECTION:
galilee.univ-paris13.fr. 7200        IN        CNAME        lb.univ-paris13.fr.
lb.univ-paris13.fr.        7200        IN        A        81.194.43.212
lb.univ-paris13.fr.        7200        IN        A        81.194.43.211
```

- Ça y est !
- `galilee.univ-paris13.fr` est un alias (`CNAME`) pour `lb.univ-paris13.fr`
- `lb.univ-paris13.fr` est associé aux adresses `81.194.43.211` et `81.194.43.212`

Victoire !

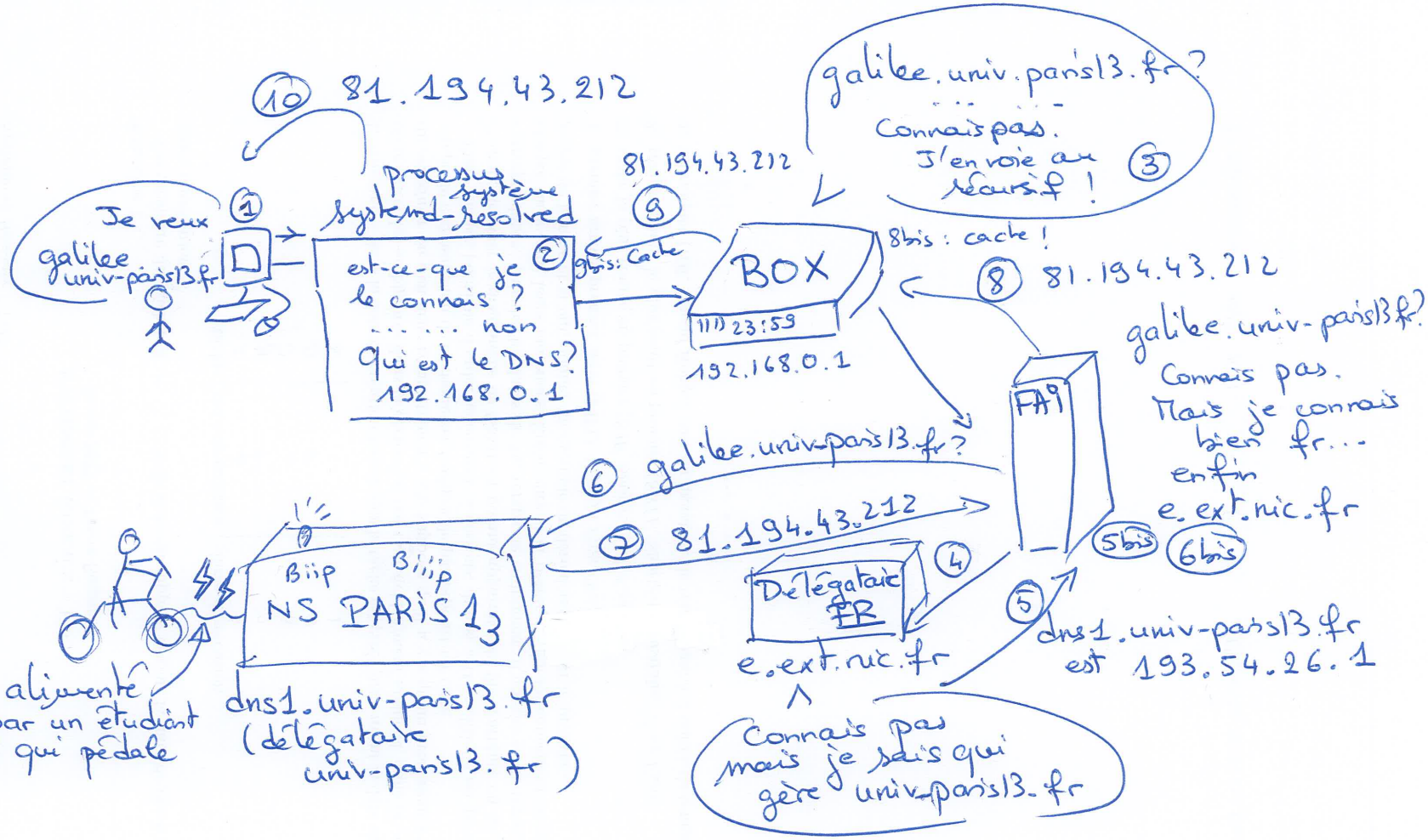
```
$ nmap 81.194.43.212
PORT    STATE SERVICE
80/tcp  open  http
443/tcp open  https
$ cat reqhttp.txt
GET / HTTP/1.1
Host: galilee.univ-paris13.fr
Accepted-Language: fr
```

```
$ cat reqhttp.txt | socat ssl:81.194.43.211:443,crlf,\
> commonname=galilee.univ-paris13.fr -
(la page index.html de galilee...)
```

Plusieurs types de serveurs DNS

Dans la vraie vie :

- Vos processus n'interrogent pas eux-mêmes les serveurs DNS qui font autorité.
- Ils commencent par faire un appel système au *résolveur du système*.
- Ce résolveur transmet la requête DNS à des serveurs DNS appelés « récursifs », mis en place par le FAI, qui font eux-mêmes ce travail (donc sont à leur tour clients des serveurs qui font autorité) et
- mémorisent (on dit « mettent en cache ») les adresses ainsi trouvées.
- Et en fait même le résolveur du système met en cache.
- Et chez vous, la *box* fait également office de résolveur intermédiaire en mettant en cache les adresses trouvées.



5bis : le récursif enregistre univ-paris13.fr dans son cache

6bis ——— galilee.univ-paris13.fr ———

8bis la box ———

9bis le résolveur du système ———

Le résolveur du système et /etc/hosts

- Sur les Linux récents (avec `systemd`), le résolveur du système est `systemd-resolved`.
- Sur tous les systèmes de type Unix, le fichier de configuration `/etc/hosts` est utilisé en priorité par le résolveur du système pour résoudre les noms de domaine.
- Par exemple,

```
$ sudo (printf '81.194.43.211 galilee.univ-paris13.fr\n' \
> >> /etc/hosts)
```

  associe cette adresse IP à ce nom de domaine sans avoir besoin de consulter quelque serveur DNS que ce soit (à part `systemd-resolved` lui-même).

Appel système au résolveur

- L'ancien appel système (que vous verrez peut-être dans vos lectures) est `gethostbyname`.
- L'interface plus moderne est donnée par `getaddrinfo` qui est un peu lourd à utiliser mais résout tous les problèmes d'adresses IP multiples (IPv4 et IPv6) et est à utiliser à partir de maintenant. [2]

2. hors programme de l'examen en 2022 – 2023 par manque de temps, voir la dernière partie de ces transparents si vous êtes intéressés

Protocole DNS

Comment un client et un serveur DNS se parlent-ils ? Ils utilisent le protocole DNS :

- au dessus d'UDP, sur le port 53
- idée générale simple : une question envoyée (un datagramme) et une réponse reçue (un datagramme)
- en-dehors de ça, nous manquons de temps pour détailler le protocole.

Le DNS

Couche réseau : IPv6

Fonction `getaddrinfo`

IPv6

- Protocole de la couche réseau, concurrent d'IPv4.
- But : simplifier et résoudre les problèmes posés par IPv4.
  - Manque d'adresses.
  - Fragmentation.
  - Options et champs non utilisés.
- Adresses sur 16 octets (4 fois plus longues qu'IPv4).
- Combien d'adresses ça fait ?

IPv6

- Protocole de la couche réseau, concurrent d'IPv4.
- But : simplifier et résoudre les problèmes posés par IPv4.
  - Manque d'adresses.
  - Fragmentation.
  - Options et champs non utilisés.
- Adresses sur 16 octets (4 fois plus longues qu'IPv4).
- Combien d'adresses ça fait ? $2^{128} \approx 3 \times 10^{38}$

IPv6

- Protocole de la couche réseau, concurrent d'IPv4.
- But : simplifier et résoudre les problèmes posés par IPv4.
  - Manque d'adresses.
  - Fragmentation.
  - Options et champs non utilisés.
- Adresses sur 16 octets (4 fois plus longues qu'IPv4).
- Combien d'adresses ça fait ? $2^{128} \approx 3 \times 10^{38}$
- Nombre d'Avogadro $\approx 6 \times 10^{23}$

Notation hexadécimale

Exemple :

- `$ ip a | grep inet6`

  ```
  inet6 ::1/128 scope host
  inet6 fe80::cf21:9a97:fa1b:4922/64 scope link
  ```

- Une adresse IPv6 de `www.wikipedia.org`

  `2a02:ec80:600:ed1a::1`

- Une adresse IPv6 de `www.google.com`

  `2a00:1450:4007:818::2003`

Notation des adresses IPv6

- Les bits sont regroupés en groupes de 16 bits (2 octets) écrits sous forme hexadécimale séparés par des `:`
- Des zéros au début peuvent être omis (par exemple le groupe `0a3b` peut être écrit `a3b`).
- Une suite (et une seule !) de tels groupes tous nuls peut être omise à l'aide de la notation `::`
- Par exemple
  - `::1` est

Notation des adresses IPv6

- Les bits sont regroupés en groupes de 16 bits (2 octets) écrits sous forme hexadécimale séparés par des `:`
- Des zéros au début peuvent être omis (par exemple le groupe `0a3b` peut être écrit `a3b`).
- Une suite (et une seule !) de tels groupes tous nuls peut être omise à l'aide de la notation `::`
- Par exemple
  - `::1` est `0:0:0:0:0:0:0:1` ou

Notation des adresses IPv6

- Les bits sont regroupés en groupes de 16 bits (2 octets) écrits sous forme hexadécimale séparés par des `:`
- Des zéros au début peuvent être omis (par exemple le groupe `0a3b` peut être écrit `a3b`).
- Une suite (et une seule !) de tels groupes tous nuls peut être omise à l'aide de la notation `::`
- Par exemple
  - `::1` est `0:0:0:0:0:0:0:1` ou `0000:0000:0000:0000:0000:0000:0000:0001`
  - `2a00:1450:4007:818::2003` est

Notation des adresses IPv6

- Les bits sont regroupés en groupes de 16 bits (2 octets) écrits sous forme hexadécimale séparés par des `:`
- Des zéros au début peuvent être omis (par exemple le groupe `0a3b` peut être écrit `a3b`).
- Une suite (et une seule !) de tels groupes tous nuls peut être omise à l'aide de la notation `::`
- Par exemple
  - `::1` est `0:0:0:0:0:0:0:1` ou `0000:0000:0000:0000:0000:0000:0000:0001`
  - `2a00:1450:4007:818::2003` est `2a00:1450:4007:818:0:0:0:2003`

Notation des adresses IPv6

- Les bits sont regroupés en groupes de 16 bits (2 octets) écrits sous forme hexadécimale séparés par des `:`
- Des zéros au début peuvent être omis (par exemple le groupe `0a3b` peut être écrit `a3b`).
- Une suite (et une seule !) de tels groupes tous nuls peut être omise à l'aide de la notation `::`
- Par exemple
  - `::1` est `0:0:0:0:0:0:0:1` ou `0000:0000:0000:0000:0000:0000:0000:0001`
  - `2a00:1450:4007:818::2003` est `2a00:1450:4007:818:0:0:0:2003`
  - `fe80::a3b::312b` est

Notation des adresses IPv6

- Les bits sont regroupés en groupes de 16 bits (2 octets) écrits sous forme hexadécimale séparés par des `:`
- Des zéros au début peuvent être omis (par exemple le groupe `0a3b` peut être écrit `a3b`).
- Une suite (et une seule !) de tels groupes tous nuls peut être omise à l'aide de la notation `::`
- Par exemple
  - `::1` est `0:0:0:0:0:0:0:1` ou `0000:0000:0000:0000:0000:0000:0000:0001`
  - `2a00:1450:4007:818::2003` est `2a00:1450:4007:818:0:0:0:2003`
  - `fe80::a3b::312b` est interdit (on comprend qu'on ne peut pas deviner la longueur des groupes omis).

Adresse IPv6

Une adresse IPv6 a une notion de *portée* (*scope*).

- La plage `::1/128` (1 seule adresse), est l'adresse de loopback. Sa portée est `hote` (*host scope*).
- La plage `fe80::/10` est l'adresse pour les communications sur la même liaison, par exemple ethernet, *link scope*.
- La plage `2000::/3` (adresses qui commencent par les bits `001`) est celle qui concernent les communications hôte à hôte (on dit *unicast*) sur le réseau global, *global unicast scope*.

Le DNS

Couche réseau : IPv6

Fonction `getaddrinfo`

Les fonctions à utiliser

```
#include <sys/socket.h>
#include <netdb.h>
int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo **res);

void freeaddrinfo(struct addrinfo *res);
const char *gai_strerror(int errcode);
```

- `getaddrinfo` pour obtenir des informations à propos d'un service sur un hôte.
- `freeaddrinfo` pour libérer toutes la mémoire
- `gai_strerror` pour traduire l'entier retourné par `getaddrinfo` en une chaîne compréhensible.

getaddrinfo

```
int getaddrinfo(const char *node, const char *service,
                const struct addrinfo *hints, struct addrinfo **res);
```

`node` est une chaîne de caractère (terminée par `'\0'`) qui désigne un hôte. Peut être

- un nom de domaine, par exemple `"www.univ-paris13.fr"`
- ou une adresse IPv4 (sous la forme d'une chaîne de caractère), par exemple `"127.0.0.1"`
- ou une adresse IPv6 (sous la forme d'une chaîne de caractère), par exemple `"::1"`

`service` est une chaîne de caractère (terminée par `'\0'`) qui désigne un service. Peut être

- un numéro de port (sous la forme d'une chaîne de caractère), par exemple `"443"` ou
- un nom de service, par exemple `"https"`.

struct addrinfo

```
struct addrinfo {
    int              ai_flags; /* pour les hints */
    int              ai_family; /* AF_INET, AF_INET6, ... */
    int              ai_socktype; /* SOCK_DGRAM, ... */
    int              ai_protocol; /* pour nous 0 */
    socklen_t        ai_addrlen; /* taille de la sockaddr */
    struct sockaddr *ai_addr; /* la sockaddr !!! */
    char            *ai_canonname; /* "nom officiel" du service */
    struct addrinfo *ai_next; /* prochaine addrinfo */
};
```

- Liste simplement chaînée !

hints

Le paramètre `const struct addrinfo *hints` sert à paramétrer les résultats renvoyés par `getaddrinfo`.
Par exemple, si l'on veut se restreindre à du TCP en IPv4,

```
struct addrinfo hints = { .ai_socktype = SOCK_STREAM,
                          .ai_family = AF_INET };
```

hints

Le paramètre `const struct addrinfo *hints` sert à paramétrer les résultats renvoyés par `getaddrinfo`.
Par exemple, si l'on veut se restreindre à du TCP en IPv4,

```
struct addrinfo hints = { .ai_socktype = SOCK_STREAM,
                          .ai_family = AF_INET };
```

Pour de l'IPv6 qui accepterait tout de même de l'IPv4 traduit (*mapped*) en IPv6 :

```
struct addrinfo hints = { .ai_flags = AI_V4MAPPED;
                          .ai_family = AF_INET6 };
```

hints

Le paramètre `const struct addrinfo *hints` sert à paramétrer les résultats renvoyés par `getaddrinfo`.
Par exemple, si l'on veut se restreindre à du TCP en IPv4,

```
struct addrinfo hints = { .ai_socktype = SOCK_STREAM,
                          .ai_family = AF_INET };
```

Pour de l'IPv6 qui accepterait tout de même de l'IPv4 traduit (*mapped*) en IPv6 :

```
struct addrinfo hints = { .ai_flags = AI_V4MAPPED;
                          .ai_family = AF_INET6 };
```

Le *mapping* d'une IPv4 sur une adresse IPv6 est, par exemple `::ffff:129.12.145.38`
Les 32 derniers bits forment l'adresse IP. Les 16 précédents sont `ffff`.
Tous les précédents sont à 0.

Résultat et valeur de retour

- Le résultat est stocké à l'adresse `res` (double-pointeur).
- La valeur de retour est 0 en cas de succès.

Séquence typique (par exemple pour du TCP) :

```
struct addrinfo *res, hints = { .ai_socktype = SOCK_STREAM };
int stat = getaddrinfo(argv[1], "1234", &hints, &res);
if (stat != 0) {
    printf("%s\n", gai_strerror(stat));
    exit(2);
}
/* on a besoin de garder la valeur de res pour libérer */
struct addrinfo *cur = res;
while (cur != NULL) {
    /* ... */
    cur = cur->ai_next;
}
freeaddrinfo(res);
```

Liste chaînée

- La fonction `getaddrinfo` appelle le résolveur du système si `node` est un nom de domaine.
- Plusieurs familles d'adresse possibles.
- Même si la famille d'adresse est fixée (par exemple à `AF_INET`), il arrive que plusieurs adresses soient retournées.
- L'idée : parcourir la liste jusqu'à réussir à se connecter (par exemple).